

Conclusión

Al implementar la clase `ExpressionTree`, busqué crear un componente robusto capaz de transformar expresiones postfijas en una estructura de árbol binario de manera ordenada. El uso de la pila como apoyo intermedio fue clave para resolver la jerarquía de las operaciones sin complicaciones sintácticas, garantizando que cada operador tome correctamente sus operandos izquierdo y derecho. Además, la inclusión de validaciones preventivas durante la construcción me aseguró evitar estados inconsistentes ante entradas mal formadas.

Para explotar al máximo el árbol una vez construido, diseñé los recorridos clásicos y presté especial atención al inorden parentizado, el cual me permite restaurar la notación infija tradicional sin perder el sentido de las prioridades aritméticas. A la par, el desarrollo de las herramientas de visualización —tanto en consola como en formato de imagen con `Graphviz`— me dio una vía clara para verificar visualmente la topología del árbol, mientras que el algoritmo de evaluación recursiva se encarga de resolver las operaciones y controlar fallos como la división por cero.

En conclusión, este proyecto me sirvió para comprender a fondo la utilidad práctica de las estructuras de datos no lineales en el procesamiento de expresiones matemáticas. Logré combinar algoritmos de recorrido, manejo de excepciones y representación gráfica en una sola clase modular, limpia y completamente funcional para cualquier análisis basado en árboles de expresión.